Electronics & Computing Club

Basic RC Car Project

Build & Code: Arduino Workshop

Written by Alexander D'Alesio on January 28th, 2026

Section 1
Overview

Would you like to take your Arduino skills to the next level by learning the fundamentals power electronics and analog circuitry? This Basic RC Car project is for you! Although the project won't be able to freely move around (or steer, for that matter), you'll learn the basics of driving motors and freely control their speed and spin direction.

The project's complexity is increased mainly due to the number of connections required to put together the circuit. One thing you will be very aware of during the project is just how much power you need to drive a motor. In fact, the Arduino is too weak to give you the power you need to drive the motors, so you'll need an external power brick to give the motors what they need. Also, you'll learn that you need a special IC called the L239D to control such a high-power element, as it can be dangerous and complicated to simply plug the motors into the Arduino. Aside from the ICs and motors, other components required include capacitors/resistors, a switch for controlling spin direction, and a potentiometer for speed adjustment.

Section 2

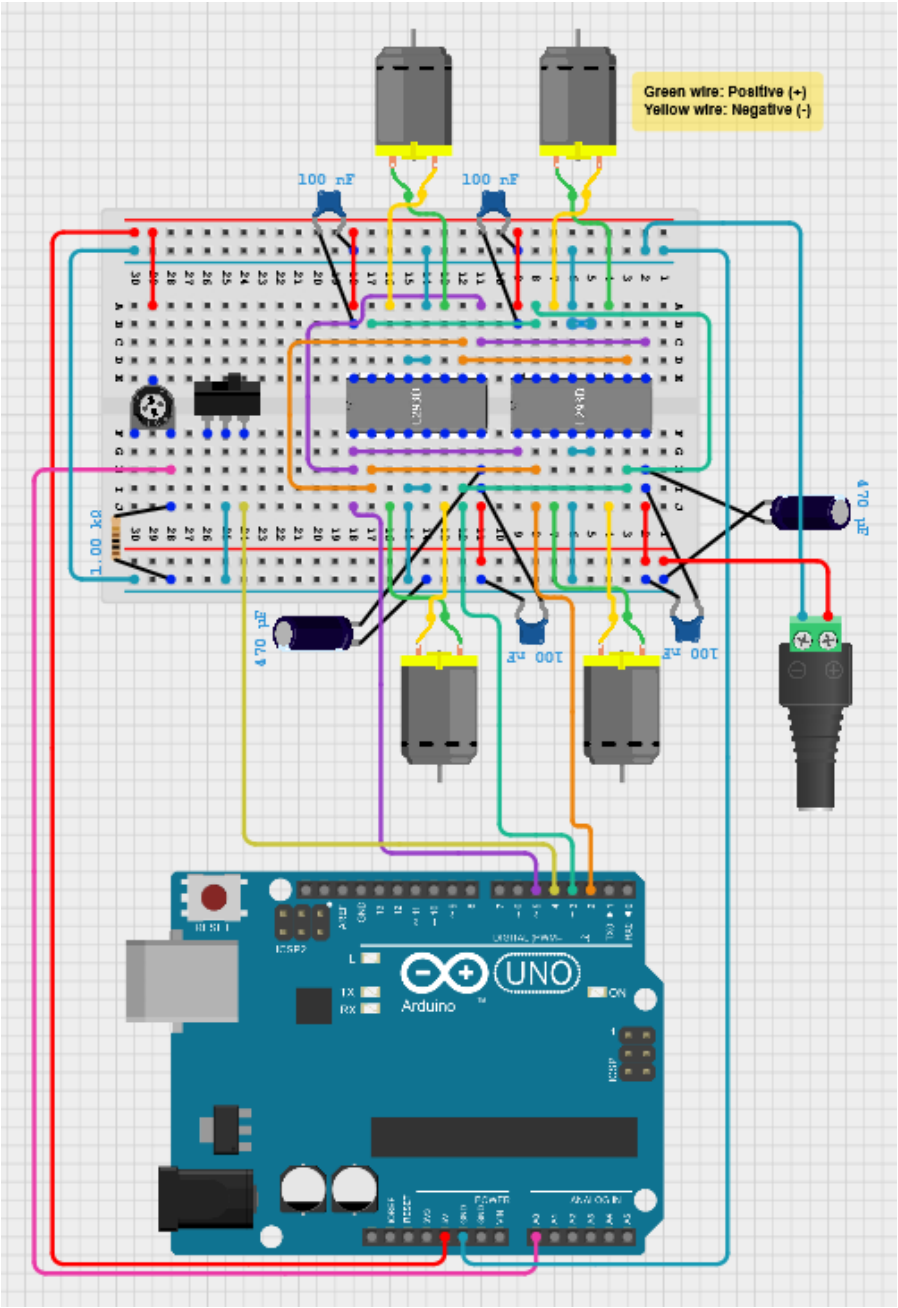
Required Components

You will need the following components:

- 4x DC Motors (3-6V)
- 4x 3D-Printed Motor Heads (disks with the black arrows)
- 2x 3D-Printed Motor Holder Jigs
- 2x L239D Quadruple Half H-Bridge Driver ICs
- 4x 0.1uF Ceramic Capacitors
- 2x 470uF Electrolytic Capacitors
- 1x 1kΩ Resistor
- 1x 1kΩ Potentiometer
- 1x Slide Switch
- 1x 5V@2A AC-DC Power Adapter (with terminal block adapter)
- 1x Arduino Uno
- Assorted Breadboard Wires
- Assorted Jumper Wires

Section 3

Circuit Assembly

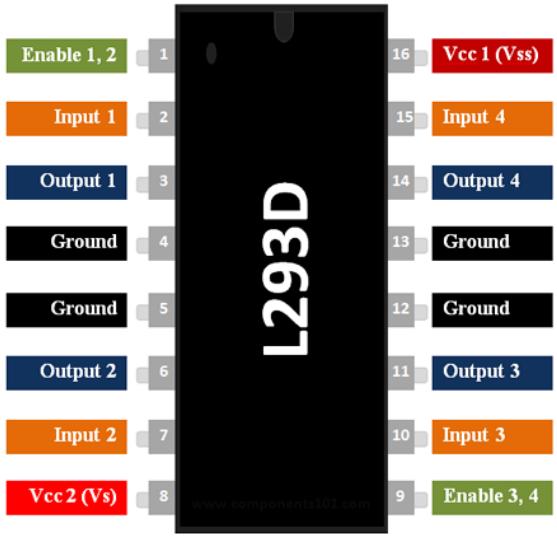


https://app.cirkitdesigner.com/project/4e15ead8-11d2-440f-87aa-91b79af40f62

The next page provides some important notes on wiring the circuit. Also, a link to the online schematic can be found under the image in case you need higher resolution.

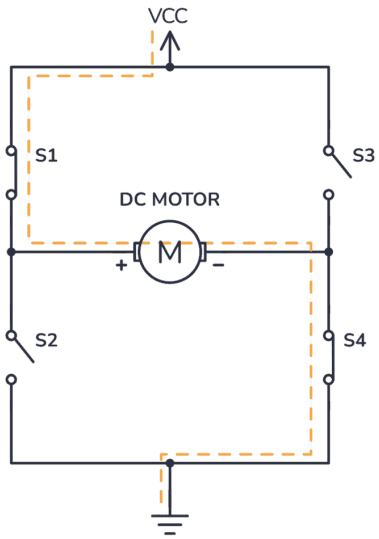
The L239D H-Bridge Driver

The L239D is a popular 16-pin, quadruple half-H driver IC designed to control the direction and speed of two DC motors (as in our case) or one stepper motor simultaneously. It's pinout is as follows:



https://components101.com/ics/l293d-pinout-features-datasheet

Essentially, an H-bridge motor driver circuit allows DC motors to run forwards, backwards, brake, and change speed by controlling the direction and intensity of current flow. Since this IC contains four half H-bridges (and one full H-bridge is required to drive a motor), we can, therefore, drive two motors. Here's a helpful schematic that visually explains how this concept works using switch theory:



https://www.build-electronic-circuits.com/h-bridge/

We can supply a PWM signal on one of it's Enable pins to control motor speed, supply a digital logic signal to two Input pins to change the spin direction, and connect a motor across the two Output pins. This configuration is mirrored on both sides of the chip, allowing us to control two motors with a single IC. Aside from these connections, the IC itself requires a 5V supply input at the Vcc pin for logic, as well as an external supply voltage on pin Vcc for powering the motor section of the IC. It needs to be an external voltage, as the Arduino 5V output pin cannot supply nearly enough current to drive these motors (unlike the logic supply, which needs less current). While you technically can control the two motors driven by this IC separately, for the sake of keeping wiring tidier, we will drive all four motors using the same PWM (speed) and direction control (Input pins) values.

To control motor direction, the configuration is as follows (notice that at no point should the inputs be simultaneously LOW or HIGH):

Input 1/3	Input 2/4	Motor Spin Direction
0	1	Counterclockwise
1	0	Clockwise

Capacitors

While they could not be very well illustrated due to space constraints on the virtual breadboard, it is very important that they are correctly added to the circuit. Two 100nF (0.1uF) ceramic capacitors need to added from ground to the logic voltage input (Vss/Pin 16) of the ICs, and the remaining two from ground to the external voltage input (Vs/Pin 8) of the ICs. The two 470uF electrolytic capacitors are also to be added across ground and the external voltage input (Vs/Pin 8) of the ICs.

Power Dissipation

Something you don't tend to think about while building circuits is the amount of heat you generate. When you're driving an LED, which takes at most 0.015 amps, you don't need to worry so much about that, but a motor passing 2 amps can be a different story altogether. The motor might not complain too much (after all, it's made to withstand that current), but remember that the ICs driving your motors are also passing that current, so they're sweating a lot more. That said, we have ways of making sure they don't overheat (without slapping a big 200$ RGB cooler on top).

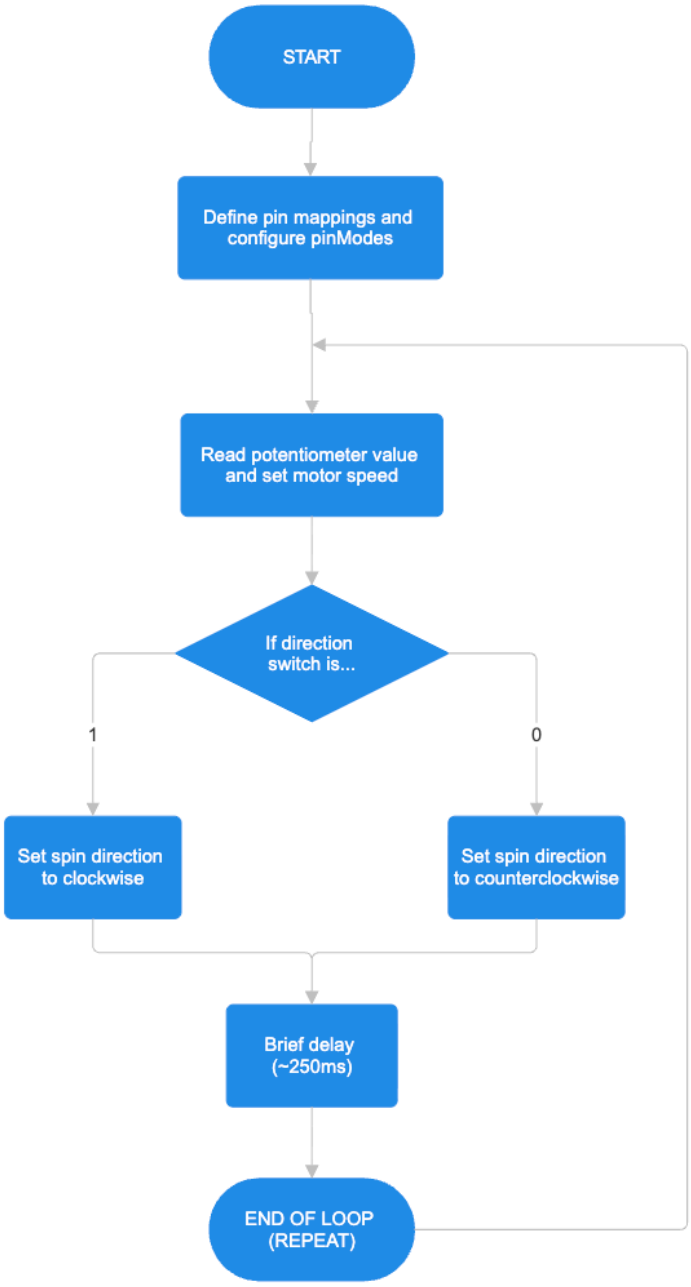
If you look at the IC's pins above, you'll notice 4 ground pins in total. It is absolutely crucial that all four are connected to ground, because they will be the cooling element. By connecting all four pins, you are giving the heat four total pathways to dissipate throughout the circuit (in this way, you can kind of think of heat as if it was electricity, where it flows through the pins from the highest to lowest temperature!).

!! WARNING !!

It is HIGHLY recommended that you do not leave the motors running for longer than 45 seconds at a time. Due to the high current in this circuit (2A), the ICs can become very hot and burn you or damage the breadboard and wires. The cooling solution described above is not the most efficient, and the circuit could potentially fail if you leave it running for too long. Be mindful of the components and what they're handling, as it could be the difference between a successful and a ruined project!

Section 4
Coding

Use this flowchart to guide your coding:



Pin Mappings

The provided skeleton code already has some pin mappings defined which we recommend you use as a best practice.

pinModes

It is up to you to determine which pinModes need to be set, although it is important that you set *DirSwitchPin* to a certain mode to avoid floating pin issues.

Map Function

You'll need to use the *map()* function to convert the raw analog potentiometer values (0-1024) into usable motor speed values (115-255). To determine the range of your specific potentiometer, it is recommended that you set up a test sketch that simply prints the analog potentiometer values to the serial monitor, turn the potentiometer to its limits, and note the values.

Data Monitoring

It is also recommended that you output the current set motor speed (converted potentiometer value) and motor spin direction to the serial monitor with every pass through the loop.

**Note that the explanations are purposely vague because this is an advanced project, and some thinking/searching is required! That being said, the code for this specific project can be written in less than 50 lines.*